Automating Secret Management with Vault Namespaces and External Secrets

Wilmer Moina & Diego Fernández

2025-04-11

Table of Contents

Chapter 1. Introduction

HashiCorp Vault is a highly flexible tool for managing secrets, encryption keys, and access policies. With the introduction of *Vault Namespaces*, organizations can isolate teams, environments, or tenants with fully independent Vault environments, all within the same cluster.

On the other hand, the *External Secrets Operator (ESO)* allows Kubernetes-native integration with external secret stores, such as Vault, AWS Secrets Manager, and others, enabling secure and declarative secret management.

This seminar explores how to:

- Provision a K3s cluster.
- Deploy Vault with multiple namespaces.
- Use External Secrets Operator to sync and inject secrets into different Kubernetes namespaces.

Chapter 2. Seminar Objectives

The main goal of this seminar is to teach students how to automate secure secret management in Kubernetes using Vault and External Secrets.

2.1. Key Outcomes

- Deploy a lightweight K3s cluster locally using `k3d`.
- Install HashiCorp Vault using Helm with support for multiple namespaces.
- Configure different Kubernetes namespaces for Dev, QA, and Prod.
- Deploy one Keycloak instance per namespace, each with its own Realm and Client.
- Use External Secrets Operator to:
  - Fetch secrets from Vault.
  - Inject them into Kubernetes workloads.
  - Automatically sync updates when Vault values change.
- Automate this setup using Terraform and Ansible.

Chapter 3. Environment and Tooling

3.1. Tools Used

- Docker (with Docker Desktop on macOS)
- k3d (K3s in Docker)
- Helm
- Terraform
- Ansible
- External Secrets Operator
- HashiCorp Vault

3.2. System Requirements

- macOS or Ubuntu with at least 8 GB RAM
- Docker installed and running
- Internet access to pull container images and charts

Chapter 4. Lab Guide

4.1. Provisioning the K3s Cluster

In this section, we will provision a local Kubernetes cluster using `k3d`, a lightweight wrapper that runs K3s inside Docker containers. This approach is particularly suitable for local development environments, hands-on labs, and CI pipelines due to its simplicity and speed.

4.1.1. Step 1: Install K3d

- For installation instructions, refer to the official k3d documentation: k3d.
- To install `kubectl`, follow the official Kubernetes documentation: kubectl.
- The Kubernetes version used in this seminar is `v1.31.5+k3s1`.

Verify the installation:

```
k3d version
```

Expected output should look similar to:

```
k3d version v5.8.3
k3s version v1.31.5-k3s1 (default)
```

4.1.2. Step 2: Create the K3s Cluster

We'll create a cluster with one server node and two agent nodes, exposing HTTPS (port 443) via the load balancer. We also disable the default Traefik ingress controller, as we will deploy our own services.

```
k3d cluster create workshop \
  --servers 1 \
  --agents 2 \
  --port "443:443@loadbalancer" \
  --k3s-arg "--disable=traefik@server:0"
```

This command:

- Creates a cluster named `workshop`.
- Starts 1 server (control-plane) and 2 agents (workers).
- Exposes port 443 from the load balancer to the host.
- Disables Traefik to avoid conflicts with custom ingress controllers.

4.1.3. Step 3: Verify Cluster Status

```
kubectl get nodes
```

Expected output:

```
NAME                    STATUS   ROLES                  AGE   VERSION
k3d-workshop-agent-0    Ready    <none>                 11s   v1.31.5+k3s1
k3d-workshop-agent-1    Ready    <none>                 11s   v1.31.5+k3s1
k3d-workshop-server-0   Ready    control-plane,master   18s   v1.31.5+k3s1
```

The K3s cluster is now ready for deploying Vault and other components.

To temporarily stop the cluster and free up system resources, you can stop the underlying Docker containers:

```
k3d cluster stop workshop
```

To bring the cluster back online later:

```
k3d cluster start workshop
```

This is especially useful during multi-day workshops or when switching between tasks, as it avoids recreating the cluster from scratch.

4.2. Installing Vault with Helm

In this step, we will install HashiCorp Vault in the Kubernetes cluster using Helm. Vault will serve as our central secret management system. We will use the official Helm chart and run it in "dev" mode initially, just to validate the deployment and make quick iterations.

Recommended Resources:

- What is Vault?
- Vault Helm chart
- Helm Installation

4.2.1. Step 1: Add the HashiCorp Helm Repository

```
helm repo add hashicorp https://helm.releases.hashicorp.com
helm repo update
```

4.3. Step 1: Deploying Vault in HA Mode

In this section, we will deploy Vault in High Availability (HA) mode using the integrated Raft storage backend. This setup provides a more realistic and production-like experience, suitable for multi-node clusters and secret replication scenarios.

We start by installing Vault using the official Helm chart, with a custom `values.yaml` file that enables HA mode and configures Raft storage with `retry_join` directives.

```
helm upgrade --install vault hashicorp/vault \
  --namespace vault \
  --create-namespace \
  -f helm/vault/values.yaml
```

The configuration:

- Deploys 3 Vault replicas.
- Enables Raft storage with automatic `retry_join`.
- Disables TLS for simplicity (not recommended for production).
- Exposes the Vault UI using a LoadBalancer service.

4.3.1. Step 2: Wait for the Pods to Be Ready

Check that all Vault pods are running:

```
kubectl get pods -n vault
```

Expected output:

```
NAME      READY   STATUS    RESTARTS   AGE
vault-0   1/1     Running   0          2m
vault-1   1/1     Running   0          2m
vault-2   1/1     Running   0          2m
```

4.3.2. Step 3: Initialize the Vault Cluster

Vault must be initialized before it can be used. This step generates the master key, which is split into multiple unseal keys.

We'll use a simplified setup with only **1 unseal key** required:

```
kubectl exec -it vault-0 -n vault -- vault operator init -key-shares=1 -key-
threshold=1
```

Vault uses Shamir's Secret Sharing algorithm to split a master key into multiple parts (shares). - `key-shares` is the total number of unseal keys generated. - `key-threshold` is the number of keys required to unseal the Vault.

For this seminar, we simplify by using 1 key out of 1 (`1-of-1`), which is **not secure for production**, but ideal for learning purposes.

Make sure to **copy and store the unseal key and root token** that are printed. You'll need them in the next step.

4.3.3. Step 4: Unseal All Vault Nodes

Each Vault pod must be manually unsealed (unless auto-unseal is configured). You will use the same unseal key generated during the initialization step.

To simplify this process, we recommend storing the unseal key in a shell variable:

```
export VAULT_UNSEAL_KEY="your-unseal-key-here"
```

Replace `"your-unseal-key-here"` with the key printed during `vault operator init`.

Then unseal each pod using:

```
kubectl exec -it vault-0 -n vault -- vault operator unseal $VAULT_UNSEAL_KEY
kubectl exec -it vault-1 -n vault -- vault operator unseal $VAULT_UNSEAL_KEY
kubectl exec -it vault-2 -n vault -- vault operator unseal $VAULT_UNSEAL_KEY
```

After all nodes are unsealed, you can verify the cluster status by running:

```
kubectl exec -it vault-0 -n vault -- vault status
```

This will show whether Vault is sealed, the current HA mode, the active node, and peer connectivity.

4.3.4. Step 5: Accessing the Vault UI

To access the Vault Web UI locally:

1. Forward the UI service port:

```
kubectl port-forward svc/vault -n vault 8200:8200
```

1. Open your browser and go to: http://localhost:8200
2. Log in using the **Initial Root Token** provided by `vault operator init`.

You now have a fully working, HA-enabled Vault deployment backed by Raft storage, ready to be extended with namespaces and External Secrets.

4.4. Preparing Kubernetes Namespaces and Workloads

Before configuring Vault, we need to prepare our Kubernetes cluster with the necessary namespaces.

Each environment (`dev`, `qa`, `prod`) will have:

- A dedicated Kubernetes namespace
- A custom ServiceAccount named `vault-sa`

This setup ensures that:

- Vault roles and policies can bind securely to service accounts per namespace
- Secrets can later be synced with External Secrets Operator (ESO) in a clean, isolated manner

4.4.1. Step 1: Create Namespaces and Service Accounts

```
for ns in dev qa prod; do
  kubectl create namespace $ns
  kubectl create serviceaccount vault-sa -n $ns
done
```

We intentionally avoid using the default service account to improve security and clarity in Vault bindings.

Once these namespaces and service accounts are in place, we can proceed to configure Vault using Terraform.

4.5. Configuring Vault

With Vault running in High Availability (HA) mode, we now configure it to manage secrets for three isolated environments: `dev`, `qa`, and `prod`.

Since Vault OSS does not support namespaces (an Enterprise-only feature), we simulate environment-level isolation using a combination of:

- Separate KV secrets engines: `secret-dev`, `secret-qa`, and `secret-prod`
- Access control policies (ACLs) scoped to each environment
- Kubernetes auth roles bound to specific namespaces

All configuration is automated using Terraform for consistency, reproducibility, and version control.

4.5.1. Directory Structure

All Terraform files related to Vault configuration are located under `terraform/vault/`:

```
terraform/vault/
├──── kubernetes_auth.tf   # Enables Kubernetes auth and defines Vault roles
├──── kv_mounts.tf         # Mounts KV v2 engines for dev, qa, and prod
├──── policies.tf          # Defines ACL policies for each environment
├──── provider.tf          # Vault provider configuration using input variables
└──── variables.tf         # Input variables for Vault address and token
```

4.5.2. Mounting KV Engines

The `kv_mounts.tf` file mounts a KV v2 secrets engine for each environment using `vault_mount` resources.

These mounts appear as:

- `secret-dev/` → used exclusively by workloads in the `dev` namespace
- `secret-qa/` → used by `qa` workloads
- `secret-prod/` → dedicated to `prod`

All engines are configured with version 2 for secret versioning and soft delete support.

4.5.3. Defining Access Policies

The file `policies.tf` defines a Vault policy for each environment. Each policy allows read-only access to the corresponding KV mount, ensuring strict separation of access.

Example: `dev` policy

```
path "secret-dev/data/*" {
  capabilities = ["read", "list"]
}
```

This approach prevents workloads in one namespace from accessing secrets belonging to another.

4.5.4. Enabling Kubernetes Authentication

The `kubernetes_auth.tf` file automates the full setup of the Kubernetes auth method in Vault:

- Enables the `kubernetes` auth backend
- Configures Vault to communicate with the in-cluster Kubernetes API
- Creates one Vault `role` per environment:
- `dev` role → bound to service accounts in the `dev` namespace
- `qa` role → bound to `qa` namespace

- `prod` role → bound to `prod` namespace

This ensures that only authorized workloads can request secrets from the correct environment.

4.5.5. Providing Vault and Kubernetes Credentials

To securely configure Vault with Kubernetes authentication, we use a `.tfvars.json` file to provide all sensitive variables required by Terraform.

Create the file `secrets.auto.tfvars.json` inside `terraform/vault/` with the following content:

```
{
  "vault_addr": "http://localhost:8200",
  "vault_token": "hvs.YourRootTokenHere",
  "kubernetes_host": "https://kubernetes.default.svc",
  "kubernetes_ca_cert": "-----BEGIN CERTIFICATE-----\nMIID...\n-----END
CERTIFICATE-----",
  "token_reviewer_jwt": "eyJhbGciOiJSUzI1NiIsIn..."
}
```

Terraform automatically loads all files ending in `.auto.tfvars.json`, so there's no need to specify this file manually during `terraform apply`.

4.5.6. How to Obtain the Required Values

These values can be extracted from any running pod in your cluster, such as `vault-0`.

To get the Kubernetes CA certificate:

```
kubectl exec -n vault vault-0 -- cat
/var/run/secrets/kubernetes.io/serviceaccount/ca.crt | awk '{printf "%s\\n", $0}'
```

Copy the entire output, including the `-----BEGIN CERTIFICATE-----` and `-----END CERTIFICATE-----` lines into the JSON value of `kubernetes_ca_cert`.

To get the token reviewer JWT:

```
kubectl exec -n vault vault-0 -- cat
/var/run/secrets/kubernetes.io/serviceaccount/token
```

Copy the full JWT token output and paste it into the `token_reviewer_jwt` field.

Do not commit the `secrets.auto.tfvars.json` file to version control. Add it to `.gitignore` to protect your credentials.

4.5.7. Applying the Configuration

Once the variables file is ready, apply the configuration:

```
cd terraform/vault
terraform init
terraform apply
```

Terraform will:

- Mount the three KV secrets engines
- Upload environment-specific access policies
- Enable and configure the Kubernetes authentication backend
- Create Vault roles bound to service accounts in the `dev`, `qa`, and `prod` namespaces

Your Vault instance is now fully configured for secure, isolated access from each Kubernetes environment.

4.5.8. Verifying Kubernetes Authentication to Vault

Now that Vault is configured, we can verify that a Kubernetes workload in the `dev` namespace can authenticate to Vault and access its secrets securely using the `vault-sa` service account.

4.5.9. Step 1: Create a test secret in Vault

First, store a secret in the `secret-dev` path using a Vault token that has the appropriate policy (e.g., the root token):

```
export VAULT_ADDR=http://127.0.0.1:8200
export VAULT_TOKEN=hvs.YourRootTokenHere

vault kv put secret-dev/example username="admin" password="s3cr3t"
vault kv put secret-qa/example username="admin" password="s3cr3t"
```

If you see the error:

```
permission denied
```

Make sure you're using a Vault token that includes the correct policy (`dev`, `qa`, or `prod`) for the secrets engine you're targeting.

Step 2: Deploy a test pod using `vault-sa`

Run the following command to deploy a temporary test pod in the `dev` namespace:

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: test-vault
  namespace: dev
spec:
  serviceAccountName: vault-sa
  containers:
  - name: tester
    image: alpine
    command: ["sh", "-c", "sleep 3600"]
  restartPolicy: Never
EOF
```

Step 3: Access the pod and install curl + jq

```
kubectl exec -it test-vault -n dev -- sh
```

Once inside the pod, install the required tools:

```
apk add --no-cache curl jq
```

Step 4: Authenticate with Vault using the pod's service account

Export the Vault address and read the pod's JWT token:

```
export VAULT_ADDR=http://vault.vault.svc:8200
export SA_TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
```

Use the token to authenticate to Vault:

```
curl --silent --request POST \
  --data '{"jwt": "'"$SA_TOKEN"'", "role": "dev"}' \
  $VAULT_ADDR/v1/auth/kubernetes/login | jq
```

Copy the `client_token` from the response.

Step 5: Use the Vault token to retrieve a secret

Now that we have a Vault token, use it to access the secret:

```
export VAULT_TOKEN=<your-client-token>
```

```
curl --silent \
  --header "X-Vault-Token: $VAULT_TOKEN" \
  $VAULT_ADDR/v1/secret-dev/data/example | jq
```

If configured correctly, you should see the secret:

```
{
  "data": {
    "data": {
      "username": "admin",
      "password": "s3cr3t"
    },
    "metadata": {
      "created_time": "2025-04-11T05:22:55.373Z",
      ...
    }
  }
}
```

Add image of the output to your lab report test:

Vault token issued via Kubernetes authentication using vault-sa in dev namespace

images::vaultTest.png[Vault token login response, width=600, align=center]

Final Step: Clean up

Exit the pod and delete it:

```
exit
kubectl delete pod test-vault -n dev
```

You can capture this output as a screenshot to include in the lab report or Medium post as proof of successful Vault-Kubernetes integration.

Chapter 5. Best Practices and Security Considerations

- Never hardcode Vault tokens or root credentials in source files.
- Use short-lived tokens and role-based access control (RBAC) in Vault.
- Ensure TLS is configured in production scenarios.
- Use namespaces to segment teams or environments cleanly.
- Set resource limits and configure alerts for secret rotation failures.

Chapter 6. References

- https://developer.hashicorp.com/vault/docs
- https://external-secrets.io
- https://k3d.io
- https://helm.sh
- https://www.keycloak.org

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
1BmE4kWBq78iYhFldvKuhfTAU6auU8tT94WrHftjDbrCEXSU1oBoqyl2QvZ6jIW3" crossorigin="anonymous">
```